

CPPRaft系列-第二篇-raft算法主要概念和主要流程

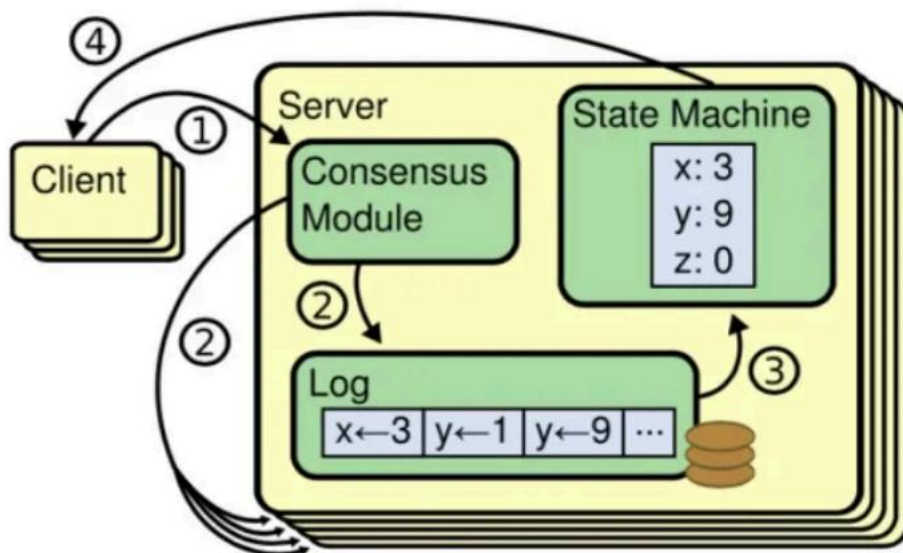
来自：代码随想录

思无邪

2023年12月17日 14:31



Raft算法



这里借用一段英文介绍：

Raft is a consensus algorithm that is designed to be easy to understand. It's equivalent to Paxos in fault-tolerance and performance. The difference is that it's decomposed into relatively independent subproblems, and it cleanly addresses all major pieces needed for practical systems. We hope Raft will make consensus available to a wider audience, and that this wider audience will be able to develop a variety of higher quality consensus-based systems than are available today.

Raft算法和Paxos算法是两种经典的经典共识算法，相比于Paxos算法，Raft算法通过拆解共识过程，引入 Leader election 等机制简化了公式过程，因此Raft算法已经是方便入门的共识算法了。

那么共识是什么呢？

共识是容错分布式系统中的一个基本问题。共识涉及多个服务器对状态机状态（对本项目而言就是上层的k-v数据库）达成一致。一旦他们对状态机状态做出决定，这个决定就是最终决定（已经被集群共识的值可以保证后面不会被覆盖，Raft的安全性）。

典型的一致性算法在其大部分服务器可用时保持运行；例如，即使有2台服务器出现故障，5台服务器的集群也可以继续运行。如果更多的服务器出现故障，它们将停止对外提供服务(但永远不会返回不正确的结果)。即小于一半的节点出现故障不会对整个集群的运行造成影响，一半或一半以上的节点出现故障则整个集群停止对外提供服务。

Raft算法和Paxos算法还有各种各样的变种（优化），网上很多资料
这里推荐 <https://raft.github.io/>，一个可视化网站，可以对Raft算法有初步的整体了解

共识算法要满足的性质

实际使用系统中的共识算法一般满足以下特性：

- 在非拜占庭条件下保证共识的一致性。非拜占庭条件就是可信的网络条件，即与你通信的节点的信息都是真实的，不存在欺骗。
- 在多数节点存活时，保持可用性。“多数”永远指的是配置文件中所有节点的多数，而不是存活节点的多数。多数等同于超过半数的节点，多数这个概念很重要，贯穿Raft算法的多个步骤。
- 不依赖于绝对时间。理解这点要明白共识算法是要应对节点出现故障的情况，在这样的环境中网络报文也很可能会受到干扰从而延迟，如果完全依靠于绝对时间，会带来问题，Raft用自定的Term（任期）作为逻辑时钟来代替绝对时间。
- 在多数节点一致后就返回结果，而不会受到个别慢节点的影响。这点与第二点联合理解，只要“大多数节点同意该操作”就代表整个集群同意该操作。对于raft来说，“操作”是储存到日志log中，一个操作就是log中的一个entry。

Raft中的一些重要概念

这里先列出涉及的所有概念，随着后续文章的继续，不清楚的概念会逐渐明了。

状态机：raft的上层应用，可以是k-v数据库（本项目）

日志、log、entry：

- 日志log：raft保存的外部命令是以日志保存
- entry：日志有很多，可以看成是一个连续的数组，而其中的的一个称为entry

提交日志commit：raft保存日志后，经过复制同步，才能真正应用到上层状态机，这个“应用”的过程称为提交

节点身份：follower、Candidate、Leader：raft集群中不同节点的身份

term：也称任期，是raft的逻辑时钟

选举：follower变成leader需要选举

领导人：就是leader

日志的term：在日志提交的时候，会记录这个日志在什么“时候”（哪一个term）记录的，用于后续日志的新旧比较

心跳、日志同步：leader向follower发送心跳（AppendEntryRPC）用于告诉follower自己的存在以及通过心跳来携带日志以同步

日志：

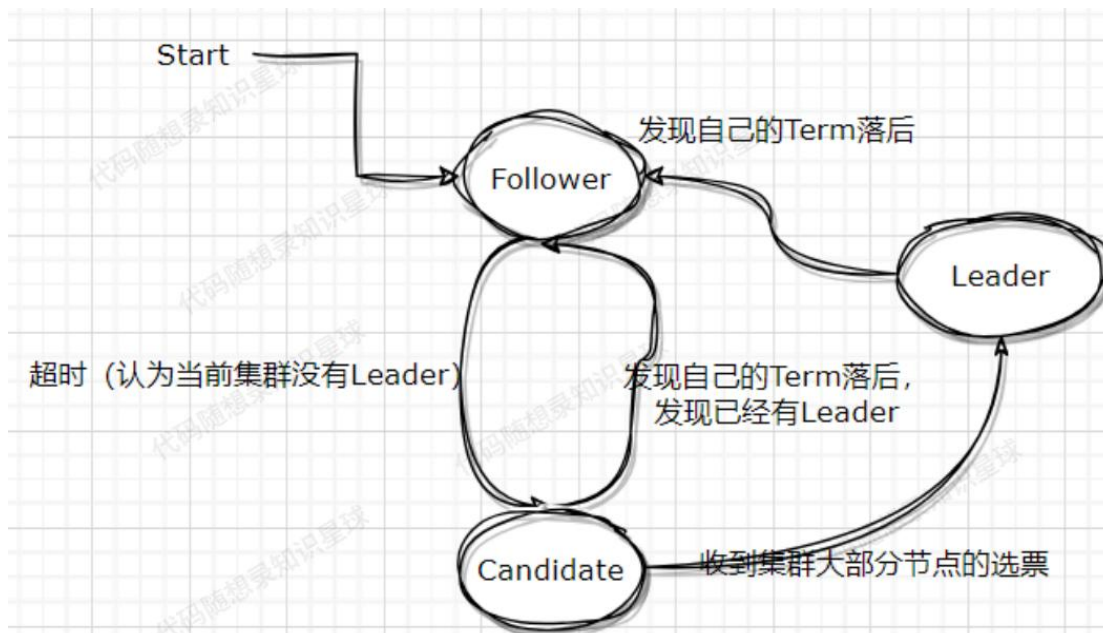
首先掌握**日志**的概念，Raft算法可以让多个节点的上层状态机保持一致的关键是让 **各个节点的日志 保持一致**，日志中保存客户端发送来的命令，上层的状态机根据日志执行命令，那么日志一致，自然上层的状态机就是一致的。

所以raft的目的就是保证各个节点的日志是相同的。

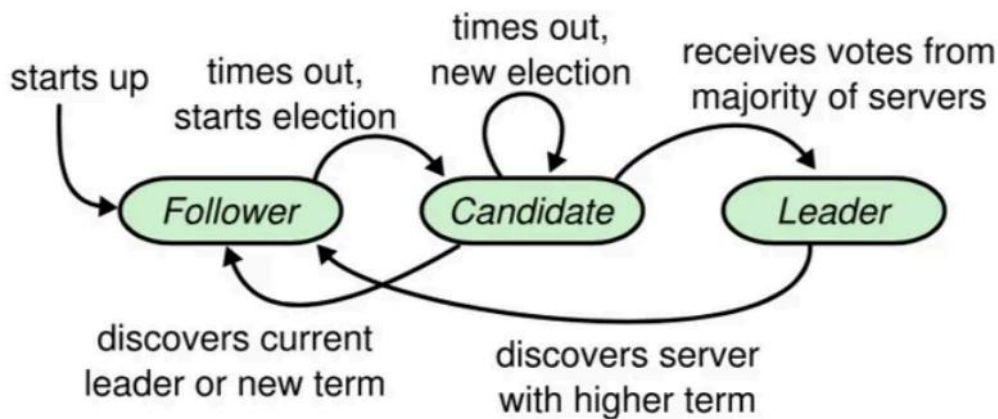
节点身份：follower、Candidate、Leader：

每个Raft节点也有自己的状态机，由下面三种状态构成：

这里的状态机与前面的状态机语义上有些“重载”，但不是一个东西，如果你无法区分，那么除了这个地方，其他地方的状态机都可以替换成 k-v数据库 以方便理解。



或者论文中的图：



- Leader：集群内最多只会会有一个 leader，负责发起心跳，响应客户端，创建日志，同步日志。
- Candidate：leader 选举过程中的临时角色，由 follower 转化而来，发起投票参与竞选。
- Follower：接受 leader 的心跳和日志同步数据，投票给 candidate。

Raft是一个强 Leader 模型，可以粗暴理解成Leader负责统领follower，如果Leader出现故障，那么整个集群都会对外停止服务，直到选举出下一个Leader。如果follower出现故障（数量占少部分），整个集群依然可以运行。

Term|任期：

Raft将Term作为内部的逻辑时钟，使用Term的对比来比较日志、身份、心跳的新旧而不是用绝对时间。Term与Leader的身份绑定，即某个节点是Leader更严谨一点的说法是集群某个Term的Leader。Term用连续的数字进行表示。Term会在follower发起选举（成为Candidate从而试图成为Leader）的时候加1，对于一次选举可能存在两种结果：

1.胜利当选：胜利的条件是超过半数的节点认为当前Candidate有资格成为Leader，即超过半数的节点给当前Candidate投了选票。

2.失败：如果没有任何Candidate（一个Term的Leader只有一位，但是如果多个节点同时发起选举，那么某个Term的Candidate可能有多位）获得超半数的选票，那么选举超时之后又会开始另一个Term（Term递增）的选举。

Raft是如何保证一个Term只有一个Leader的？

- 因为Candidate变成Leader的条件是获得超过半数选票，一个节点在一个Term内只有一个选票（投给了一个节点就不能再投递给另一个节点），因此不可能有两个节点同时获得超过半数的选票。

发生故障时，一个节点无法知道当前最新的Term是多少，在故障恢复后，节点就可以通过其他节点发送过来的心跳中的Term信息查明一些过期信息。

当发现自己的Term小于其他节点的Term时，这意味着“自己已经过期”，不同身份的节点的处理方式有所不同：

- leader、Candidate：退回follower并更新term到较大的那个Term
- follower：更新Term信息到较大的那个Term

这里解释一下为什么 自己的Term小于其他节点的Term时leader、Candidate会退回follower 而不是延续身份，因为通过Term信息知道自己过期，意味着自己可能发生了网络隔离等故障，那么在此期间整个Raft集群可能已经有了新的leader、**提交了新的日志**，此时自己的日志是有缺失的，如果不退回follower，那么可能会导致整个集群的日志缺失，不符合安全性。

相反，如果发现自己的Term大于其他节点的Term，那么就会忽略这个消息中携带的其他信息。

安全性：

- Election Safety ：每个 term 最多只会会有一个 leader；集群同时最多只会会有一个可以读写的 leader。

每个Term最多只会会有一个leader在前面已经解释过；集群同时最多只会会有一个可以读写的 leader 是指集群中由于发生了网络分区，一个分区中的leader会维护分区的运行，而另一个分区会因为没有leader而发生选举产生新的leader，任何情况下，最多只有一个分区拥有绝大部分节点，那么只有一个分区能写入日志，这个在[共识算法要满足的性质]一节中已经介绍过。

- Leader Append-Only ：leader 的日志是只增的。
- Log Matching ：如果两个节点的日志中有两个 entry 有相同的 index 和 term，那么它们就是相同的 entry。

这是因为在Raft中，每个新的日志条目都必须按照顺序追加到日志中。

在运行过程中会根据这条性质来检查follower的日志与leader的日志是否匹配，如果不匹配的话leader会发送自己的日志去覆盖follower对应不匹配的日志。

- Leader Completeness ：一旦一个操作被提交了，那么在之后的 term 中，该操作都会存在于日志中。
- State Machine Safety ：状态机一致性，一旦一个节点应用了某个 index 的 entry 到状态机，那么其他所有节点应用的该 index 的操作都是一致的。

想检验自己对于状态转换是否掌握建议在阅读本文后浏览raft 论文的图 2，检验是否对其中的每一句话都理解了。

Raft中的一些重要过程

领导人选举相关：

前面提到：Raft是一个强 Leader 模型，可以粗暴理解成Leader负责统领follower，如果Leader出现故障，那么整个集群都会对外停止服务，直到选举出下一个Leader。

很自然的问题是：节点之间通过网络通信，其他节点（follower）如何知道leader出现故障？follower知道leader出现故障后如何选举出leader？符合什么条件的节点可以成为leader？

节点之间通过网络通信，其他节点（follower）如何知道leader出现故障？

leader会定时向集群中剩下的节点（follower）发送AppendEntry（作为心跳，heartbeat）以通知自己仍然存活。

可以推知，如果follower在一段时间内没有接收leader发送的AppendEntry，那么follower就会认为当前的leader出现故障，从而发起选举。

这里“follower在一段时间内没有接收leader发送的AppendEntry”，在实现上可以用一个定时器和一个标志位来实现，每到定时时间检查这期间有无AppendEntry即可。

AppendEntry 具体来说有两种主要的作用和一个附带的作用：

主要作用：

- 心跳
- 携带日志entry及其辅助信息，以控制日志的同步和日志向状态机提交

附带的作用：

- 通告leader的index和term等关键信息以便follower对比确认follower自己或者leader是否过期

follower知道leader出现故障后如何选举出leader？

参考【节点身份： follower、Candidate、Leader】一节中的描述，follower认为leader故障后只能通过：term增加，变成candidate，向其他节点发起RequestVoteRPC申请其他follower的选票，过一段时间之后会发生如下情况：

- 赢得选举，马上成为 leader（此时term已经增加了）

发现有符合要求的leader，自己马上变成follower了，这个符合要求包括：leader的term $\geq$ 自己的term

- 一轮选举结束，无人变成leader，那么循环这个过程，即：term增加，变成candidate，。。。。

赢得选举的条件前面也有过提及，即获得一半以上的选票。

不妨思考和回顾下如下两个问题：

- 如果在选举过程中没有“一半以上”选票的限制，会发生什么？会有什么问题？
- raft节点的数量要求是奇数，为什么有这个要求？
- 如果发现一个leader，但是其term小于自己会发生什么？

为了防止在同一时间有太多的follower转变为candidate导致一直无法选出leader，Raft采用了随机选举超时（randomized election timeouts）的机制，每一个candidate在发起选举后，都会随机化一个新的选举超时时间。

符合什么条件的节点可以成为leader？

这一点也成为“选举限制”，有限制的目的是为了保证选举出的leader一定包含了整个集群中目前已committed的所有日志。

当 candidate 发送 RequestVoteRPC 时，会带上最后一个 entry 的信息。所有的节点收到该请求后，都会比对自己的日志，如果发现自己的日志更新一些，则会拒绝投票给该 candidate，即自己的日志必须要“不旧于”改candidate。

判断日志老旧的方法raft论文中用了一段来说明，这里说一下如何判断日志的老旧：

需要比较两个东西：最新日志entry的term和对应的index。index即日志entry在整个日志的索引。

```
if 两个节点最新日志entry的term不同
    term大的日志更新
else
    最新日志entry的index大的更新
end
```

这样的限制可以保证：成为leader的节点，其日志已经是多数节点中最完备的，即包含了整个集群的所有 committed entries。

日志同步、心跳：

在RPC中 日志同步 和 心跳 是放在一个RPC函数（AppendEntryRPC）中来实现的，原因为：

- 心跳RPC 可以看成是没有携带日志的特殊的 日志同步RPC。

对于一个follower，如果leader认为其日志已经和自己匹配了，那么在AppendEntryRPC中不用携带日志（再携带日志属于无效信息了，但其他信息依然要携带），反之如果follower的日志只有部分匹配，那么就需要在AppendEntryRPC中携带对应的日志。

很自然的问题是：感觉很复杂，为什么不直接让follower拷贝leader的日志？leader如何知道follower的日志是否与自己完全匹配？如果发现不匹配，那么如何知道哪部分日志是匹配的，哪部分日志是不匹配的呢？

感觉很复杂，为什么不直接让follower拷贝leader的日志|leader发送全部的日志给follower？

leader发送日志的目的是让follower同步自己的日志，当然可以让leader发送自己全部的日志给follower，然后follower接收后就覆盖自己原有的日志，但是这样就会携带大量的无效的日志（因为这些日志follower本身就有）。

因此 raft的方式是：先找到日志不匹配的那个点，然后只同步那个点之后的日志。

leader如何知道follower的日志是否与自己完全匹配？

在AppendEntryRPC中携带上 entry的index和对应的term（日志的term），可以通过比较最后一个日志的index和term来得出某个follower日志是否匹配。

如何根据某个日志的index和其term来得出某个日志是否匹配在【raft日志的两个特点】一节中。

如果发现不匹配，那么如何知道哪部分日志是匹配的，哪部分日志是不匹配的呢？

leader每次发送AppendEntryRPC后，follower都会根据其entry的index和对应的term来判断某一个日志是否匹配。

在leader刚当选，会从最后一个日志开始判断是否匹配，如果匹配，那么后续发送AppendEntryRPC就不需要携带日志entry了。

如果不匹配，那么下一次就发送 倒数第2个 日志entry的index和其对应的term来判断匹配，

如果还不匹配，那么依旧重复这个过程，即发送 倒数第3个 日志entry的相关信息

重复这个过程，知道遇到一个匹配的日志。

raft日志的两个特点

raft对于日志可以保证其具有两个特点：

- 两个节点的日志中，有两个 entry 拥有相同的 index 和 term，那么它们一定记录了相同的内容/操作，即两个日志匹配
- 两个节点的日志中，有两个 entry 拥有相同的 index 和 term，那么它们前面的日志entry也相同

如何保证这两点：

- 保证第一点：仅有 leader 可以生成 entry
- 保证第二点：leader 在通过 AppendEntriesRPC 和 follower 通讯时，除了带上自己的term等信息外，还会带上 entry的index和对应的term等信息，follower在接收到后通过对比就可以知道自己与leader的日志是否匹配，不匹配则拒绝请求。leader发现follower拒绝后就知道entry不匹配，那么下一次就会尝试匹配前一个entry，直到遇到一个entry匹配，并将不匹配的entry给删除（覆盖）。

注意：raft为了避免出现一致性问题，要求 leader 绝不会提交过去的 term 的 entry（即使该 entry 已经被复制到了多数节点上）。leader 永远只提交当前 term 的 entry，过去的 entry 只会随着当前的 entry 被一并提交。

优化：寻找匹配加速（可选）

在寻找匹配日志的过程中，在最后一个日志不匹配的话就尝试倒数第二个，然后不匹配继续倒数第三个。。。

leader和follower 日志存在大量不匹配的时候这样会太慢，可以用一些方式一次性的多倒退几个日志，就算回退稍微多了几个也不会太影响，具体实现参考[7.3 快速恢复 \(Fast Backup\) - MIT6.824 \(gitbook.io\)](#)

后续代码实现时也会使用类似的加速方法，到时候也会再简单的介绍一下。

总结

本节主要讲解了raft算法的主要概念和主要流程，如果流程中有什么问题没有讲清 欢迎大家提问。

下一节的主要内容应该是关于raft的主要流程的函数实现，目前考虑的方法是给出关键函数后对函数进行讲解。

此外，关于日志压缩（快照，snapshot）的部分与raft主要流程并不相关，这里考虑在后续再进行讲解。

